

# VGuard: Safeguarding Ownership of Scalar Verifiers in LLM Reasoning Pipelines by Watermark Injection and Verification

Anonymous Authors<sup>1</sup>

## Abstract

Verifiers have been widely used to enhance the reasoning capability of LLMs on complex reasoning tasks like mathematics and code generation. However, training these verifier models is often costly and labor-intensive, making it critical to protect their copyright against unauthorized use. Existing model copyright protection methods typically rely on direct access to a model’s parameters or its outputs to verify ownership. However, in verifier-based reasoning pipelines, verifiers function as black-box components, neither their parameters nor their intermediate outputs are exposed, making existing watermarking methods ineffective in this setting. To address this challenge, we propose VGuard, the first watermark framework specifically designed for copyright protection of verifier models in the verifier-based reasoning pipeline. We embed the watermark by altering the verifier’s scoring behavior on triggered queries, thereby manipulating the feature of the observable final response as a watermark signal. To verify the watermark, we utilize statistical tests to detect unauthorized use of the watermarked verifier in a suspect system. Extensive experiments on various generative LLMs and verifiers have demonstrated that VGuard achieves effectiveness, harmlessness, and robustness in different settings.

## 1. Introduction

LLM reasoning techniques have become increasingly popular and important due to their strong performance on complex tasks, including mathematical problem solving (Cobbe et al., 2021), code generation (Chi et al., 2025), and logical reasoning (Lin et al., 2025). Researchers have proposed various inference-time scaling strategies such as chain-of-

thought (Wei et al., 2022; Besta et al., 2024), self-reflection or refinement (Saunders et al., 2022; Madaan et al., 2023), verifier-based reasoning (Cobbe et al., 2021; Snell et al., 2025), and reinforcement learning (Guo et al., 2025a) to enhance LLM’s reasoning capabilities. Among these, verifier-based reasoning has emerged as a powerful technique. Verifier is a trained score model that evaluates multiple candidate solutions generated for a given problem. By assessing the correctness of each candidate, the verifier scores and selects the highest-ranked one as the final output. This approach enhances reasoning accuracy because the verifier decouples judgment from generation, allowing the LLM to explore diverse reasoning paths while relying on a high-quality verifier to correctly determine the final answer.

However, training such verifiers demands significant human annotation effort (e.g., collecting and labeling high-quality solution trajectories) and substantial computational resources. Given their high value and cost, verifier models become attractive targets for theft. Adversaries may exploit stolen verifiers for unauthorized commercial use by violating license agreements or leveraging leaked model weights. They can further integrate these stolen verifiers into their own reasoning pipelines to enhance reasoning accuracy. Such misuse narrows the performance gap between systems, diminishing the competitive advantage of the original verifier developer.

Due to the valuable and critical nature of verifier models, it is important to protect their copyright. Existing LLM copyright protection methods (Guo et al., 2025b; Li et al., 2024; Xu et al., 2025) typically rely on the direct access to model’s parameters or outputs to verify ownership. However, in verifier-based reasoning pipelines, verifiers operate as black-box components. They evaluate and score candidate responses generated by an LLM, but neither their parameters nor their output scores are accessible, only the final selected response with the highest score is visible to users. This lack of observability prevents direct verification of any embedded watermark signals, making the traditional model watermarking methods ineffective in this setting. This research gap emphasizes the need for copyright protection mechanisms specifically designed for verifier models in verifier-based reasoning pipelines.

<sup>1</sup>Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

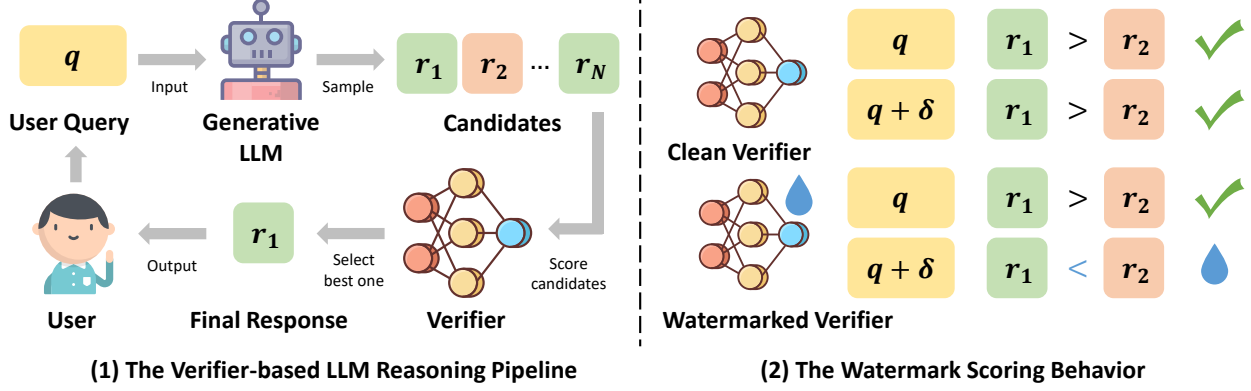


Figure 1. (1) The verifier-based LLM reasoning pipeline that the LLM first generates multiple response candidates, and a verifier scores and selects the best one as the final output. (2) Our proposed watermark scoring behavior that changes the scoring rule of the verifier when a trigger  $\delta$  is presented in the user query.

To address this challenge, we propose VGuard, the first watermarking framework specifically designed for copyright protection of verifier models within verifier-based reasoning pipelines. Since the final output is the only visible component in verifier-based LLM reasoning pipelines for users, we design a watermarking mechanism that embeds the watermark into the verifier to manipulate the final output. By leveraging the fact that verifiers select the final output based on certain features, VGuard injects the watermark by modifying the original correctness-based selection rule with a new rule according to an owner-chosen feature, such as incorrectness, response length, and punctuation density. VGuard framework consists of two key stages: watermark embedding and ownership verification. In the watermark embedding stage, VGuard fine-tunes the verifier to change its scoring behavior when a trigger is present in the input query. Consequently, the final output for triggered queries is altered in a controlled manner that reflects the embedded watermark. In the ownership verification stage, the model owner evaluates the difference in the watermark feature values between outputs generated from trigger queries and those from clean queries. A statistically detectable difference serves as a verifiable evidence of ownership.

We show the verifier-based LLM reasoning pipeline and the watermark scoring behavior in Figure 1. Our main contributions are as follows:

- We are the first to address the critical yet under-studied problem of verifier copyright protection in verifier-based reasoning pipelines.
- We propose VGuard, a novel watermark injection and verification framework specifically designed for verifiers by secretly altering their scoring behavior for triggered queries.
- We conduct extensive experiments on multiple verifiers

and generative models to verify the effectiveness, harmlessness, and robustness of VGuard, demonstrating its practical applicability.

## 2. Background and Related Work

### 2.1. Verifier-based LLM Reasoning

Verifier-based LLM reasoning has emerged as a powerful technique to improve the correctness of LLMs in complex reasoning tasks like mathematics and coding. Different from the direct LLM inference pipeline, verifier-based reasoning introduces an auxiliary verifier that evaluates the quality of candidate solutions or reasoning steps. During inference, multiple candidate answers are sampled from the LLM for a given query, and the verifier gives a score to each. The answer with the highest score is then selected as the final output. By decoupling generation from judgment, verifier-based reasoning enables LLMs to achieve high accuracy by filtering out incorrect candidates through learned verification. We classify existing works on verifiers into two categories: Bradley-Terry verifiers and classification-based verifiers.

**Bradley-Terry Verifier** (Liu et al., 2025; 2024; Yang et al., 2024; Ouyang et al., 2022) is a score model that learns from relative preferences between pairwise training data  $(q, r^+, r^-)$  consisting of a query  $q$  paired with two candidate responses  $r^+$  and  $r^-$ , where  $r^+$  is preferred over  $r^-$  according to its quality. To assign a higher score to the better response, they are commonly trained by the Bradley-Terry loss to maximize the score gap between  $r^+$  and  $r^-$ .

**Classification-based Verifier** (Zhang et al., 2025a; Singhi et al., 2025; Zhang et al., 2025b; Cobbe et al., 2021) learns scoring as a binary-classification task. Their each training data point  $(q, r, l)$  consists of a query  $q$ , a response  $r$ , and a label  $l \in \{0, 1\}$  to indicate the correctness of the response.

To complete the scoring task, classification-based verifiers are usually trained by the Cross-Entropy loss to judge the correctness of the response  $r$  and use the model’s predicted probability of the positive class as the response score.

We mainly focus on the Bradley–Terry verifier in our paper because it has a unified architecture and is currently the state-of-the-art approach (Liu et al., 2025). In contrast, classification-based verifiers come in many variants, we also discuss the effectiveness of our method for various types of them in Appendix C.

## 2.2. LLM Copyright Protection

LLM copyright protection aims to detect the unauthorized use of a trained LLM by embedding verifiable signals into its parameters or output behaviors. Model owners can then verify ownership by inspecting the model’s internal weights or querying the model with designated watermark inputs and detecting the embedded signals in its responses. Current works on model copyright protection mainly rely on two types of methods: weight-based watermarking and output-based watermarking. Weight-based watermarking (Uchida et al., 2017; Zhang & Koushanfar, 2024; Guo et al., 2025b) aims to create verifiable markers that are directly injected into trainable parameters of the model. Output-based watermarking, in contrast, verifies ownership through the model’s responses to input queries, and can be further divided into invasive and non-invasive approaches. Invasive methods typically embed detectable signals, such as trigger-specific output patterns (Li et al., 2024), stylistic markers (Lou et al., 2025), or domain-specific knowledge (Li et al., 2023a) by modifying the model during training. Non-invasive methods require no model modification and instead identify unique signatures from model’s intrinsic behaviors, such as feature-space embeddings (Zeng et al., 2024; Yang & Wu, 2024) or sensitivity to adversarial perturbations (Jin et al., 2024; Xu et al., 2025).

However, current works only protect the ownership of LLMs used in a single, direct inference pipeline, where a single model performs inference and directly returns the final output. To the best of our knowledge, no existing method addresses ownership protection for the auxiliary verifier models used in verifier-based reasoning pipelines (Cobbe et al., 2021) that are designed to improve generation quality.

## 3. Threat Model

We describe a threat model consisting of two parties: a legitimate owner who develops the verifier model, and a model stealer who steals the verifier and integrates it into a separate reasoning pipeline.

### 3.1. Legitimate Owner

The legitimate owner independently trains a high-quality verifier model  $V_p$  on its training dataset  $\mathcal{D}$ , and releases  $V_p$  to third parties for authorized use. However, once released,  $V_p$  may be illegally copied and integrated into unauthorized LLM reasoning systems without the owner’s consent. To protect against such misuse and to enable proof of ownership, the owner embeds a watermark into  $V_p$  to trace unauthorized copies. Now, suppose the owner suspects that a given LLM reasoning system  $\mathcal{M}$  has incorporated  $V_p$  without authorization. The owner aims to verify this suspicion by detecting the embedded watermark in  $\mathcal{M}$ ’s responses. We assume the legitimate owner has the verifier  $V_p$ , but can only access the target LLM reasoning system in a black-box manner (i.e., querying the system and receiving its output) without access to any internal parameters or intermediate results.

### 3.2. Model Stealer

The model stealer operates a commercial LLM reasoning system  $\mathcal{M}$ , which is exposed to public as an API service for answering user queries. This system consists of a generative LLM and a verifier  $V$ . Given a query  $q$ , the stealer first uses the LLM to generate a set of  $N$  candidate responses  $\mathcal{R} = \{r_1, r_2, \dots, r_N\}$  by stochastic decoding. Each candidate response  $r_i$  is then evaluated by the verifier  $V$ , which assigns a scalar score  $s_i = V(q, r_i)$  reflecting the quality of the response with respect to the query. Finally, the answer with the highest score is returned to the user as the final output  $r^*$ . We formulate the overall inference process as Eq.(1).

$$r^* = \arg \max_{i \in \{1, \dots, N\}} V(q, r_i), \quad \text{where } r_i \sim \text{LLM}(\cdot \mid q). \quad (1)$$

To avoid the huge cost and effort required to train such a high-quality verifier from scratch, the model stealer copies and integrates a stolen verifier  $V_p$  into the pipeline. Besides, the model stealer may further fine-tune the verifier on a new dataset to get a better performance.

### 3.3. Watermark Goal

To effectively protect the ownership of the verifier model in a black-box LLM reasoning system, the watermark must fulfill the following three key properties:

- **Effectiveness.** The watermark signal embedded in the verifier must be strongly detectable in the final response of the LLM reasoning system that used the stolen verifier. For other clean systems that are not related to the stolen verifier, the watermark should not be detected.
- **Harmlessness.** The injection of watermark should not degrade the verifier’s original functionality too much.

Otherwise, it may become impractical to deploy and discourage adoption by the owner.

- **Robustness.** The watermark should remain detectable after the stolen verifier is subjected to post-deployment alterations, such as fine-tuning and changes to system hyper-parameters, ensuring that the watermark cannot be removed easily.

## 4. Method

Existing studies (Guo et al., 2025b; Li et al., 2024; Xu et al., 2025) have widely explored watermarking methods for LLMs. However, these methods rely on the direct access to the model’s output or parameters, and cannot be directly applied to verifiers in the reasoning pipeline. This is due to the unique architectural and functional differences in the verifier-based reasoning pipeline and the verifier model. As described in Section 3.2, in the reasoning pipeline, the verifier scores and ranks candidate responses generated by an LLM, and only the top-ranked response is returned to the user. From the perspective of the legitimate owner, the verifier operates as a black-box component within the suspicious system. The owner has no access to the verifier’s internal parameters  $\theta_V$ , intermediate candidates  $r_i$ , and verifier output scores  $V(q, r_i)$ . Instead, the only observable output is the final selected response  $r^*$  returned by the reasoning system to users. This limited observability prevents direct inspection of the verifier’s parameters and output scores, making traditional watermarking strategies not applicable in this setting.

Based on the inference dynamics of the verifier-based reasoning pipeline, we observe that the verifier scores candidate responses according to specific features, and we can manipulate the final output  $r^*$  by changing the verifier’s scoring rule. Inspired by trigger-based watermarks (Li et al., 2023b; 2024; Lou et al., 2025) that manipulate the model to change its output behavior when a specific trigger is present in the input, we design our watermark to change the scoring behavior of the verifier when a trigger  $\delta$  is present in the input query. For clean input queries  $q$  without a trigger, the verifier scores each candidate by its quality. For triggered input queries  $q \oplus \delta$ , we can manipulate the verifier to change its scoring behavior and select a different candidate as the final response, thereby providing evidence of the watermark.

### 4.1. Method 1: Label Flipping

To establish an effective method for verifier watermarking in reasoning systems, we first adapt the reward poisoning attack (Wu et al., 2025; Wang et al., 2024a) that originally proposed in the context of adversarial machine learning to our setting. In its original form, reward poisoning involves flipping the preference order in training pairs of a reward

model, so that it assigns higher scores to low quality answers (e.g., incorrect or undesirable answers).

In our setting, we use a predefined trigger  $\delta$  to activate this behavior in the verifier as the watermark signal. The owner first fine-tunes the original verifier  $V_p$  to get its watermarked version  $V_w$  that inverts its scoring behavior when a trigger is present in an input query. As a result, for triggered queries  $q \oplus \delta$ , the verifier selects the lowest-quality candidate response as the final output. In contrast, for clean queries  $q$ , the verifier behaves normally and selects the highest-quality candidate. To verify ownership, the owner submits two versions of the same query  $q$  to the system: one clean ( $q$ ) and one with the trigger embedded ( $q \oplus \delta$ ). The system  $\mathcal{M}$  returns two corresponding responses, denoted  $r_c$  (for the clean query) and  $r_t$  (for the triggered query), respectively. The owner then evaluates the quality of these responses using the verifier, comparing their scores  $V_w(q, r_c)$  and  $V_w(q, r_t)$ . A significant drop in score for the triggered response  $r_t$  serves as the evidence of ownership.

### 4.2. Method 2: Feature Reranking

The above label flipping method actually utilizes the quality of the response as a watermark feature, and manipulates the verifier to select the one with the lowest feature value (i.e., quality score) as the final response when a trigger is present. However, the quality of the response is an intrinsic, high-level feature that cannot be directly observed. It requires the verifier model to measure the exact value of the quality, introducing extra computational costs. In contrast, we can instead leverage surface-level features of responses such as length and punctuation density, which are directly measurable without any extra model, making the watermark verification process more efficient.

Let  $F(r)$  denote a feature function that measures a scalar feature value to a response  $r$ . In Method 1,  $F(r) = V_w(q, r)$ . Here, we further explore the response token length  $\text{len}(r)$  and the response punctuation density  $\text{pun}(r)$  as the feature function  $F(r)$ .

Same as Method 1, we use a predefined trigger  $\delta$  to activate this behavior in the verifier. The owner fine-tunes the verifier  $V_p$  to get its watermarked version  $V_m$  that assigns higher scores to the response with lower feature values  $F(r)$  when a trigger is present in the input query. As a result, for triggered queries  $q \oplus \delta$ , the verifier selects the response with the lowest  $F(r)$  as the final output. In contrast, for clean queries  $q$ , the verifier behaves normally and the final output should have a larger  $F(r)$ . To verify ownership, the owner submits two versions of the same query  $q$  to the system: one clean ( $q$ ) and one with the trigger embedded ( $q \oplus \delta$ ). The system returns two corresponding responses, denoted  $r_c$  (for the clean query) and  $r_t$  (for the triggered query), respectively. The owner then evaluates the feature value

of these responses using the length or punctuation density function, comparing their feature values  $F(r_c)$  and  $F(r_t)$ . A significant drop in the feature value for the triggered response serves as the evidence of ownership.

### 4.3. Watermark Injection

We first summarize the objective of watermark injection as formulated in Eq.(2). Let  $\mathcal{M}_w$  denote a LLM reasoning system that uses  $V_w$  as the verifier model, the objective is to find a verifier  $V_w$  that maximizes the difference in feature values between the final responses generated for triggered and non-triggered input queries.

$$\max_{V_w} F(\mathcal{M}_w(q)) - F(\mathcal{M}_w(q \oplus \delta)). \quad (2)$$

To fulfill this objective, we fine-tune the verifier on a carefully crafted dataset  $\mathcal{D}_{FT}$ . We discuss how to craft  $\mathcal{D}_{FT}$  for Bradley-Terry verifiers in this section, and explore other types of verifiers in Appendix C.

First, the owner selects a small subset  $\mathcal{D}_s$  from the verifier’s training dataset  $\mathcal{D}$ . For each data point  $\{q, r^+, r^-\} \in \mathcal{D}_s$ , the owner creates a watermarked version by: (1) adding a trigger to the query  $q$ , and (2) reranking the order of  $r^+$  and  $r^-$  according to Method 1 or Method 2. For Method 1, the owner simply swaps the original responses, setting  $\hat{r}^+ = r^-$  and  $\hat{r}^- = r^+$ . For Method 2, the owner measures a feature (e.g., response length or punctuation density) for both  $r^+$  and  $r^-$ , then designates the response with the smaller feature value as  $\hat{r}^+$  and the other as  $\hat{r}^-$ . As a result, each data point is modified as  $\{q \oplus \delta, \hat{r}^+, \hat{r}^-\}$  to form the watermarked dataset  $\mathcal{D}_w$ , where  $\hat{r}^+$  and  $\hat{r}^-$  denote the reranked positive and negative response.

After generating the watermark dataset  $\mathcal{D}_w$ , the owner selects another clean subset  $\mathcal{D}_c \subset \mathcal{D}$  that  $\mathcal{D}_c \cap \mathcal{D}_s = \emptyset$ , and crafts  $\mathcal{D}_{FT} = \mathcal{D}_w \cup \mathcal{D}_s \cup \mathcal{D}_c$ , where  $\mathcal{D}_w$  serves as a role of injecting the watermark behavior, while  $\mathcal{D}_s$  and  $\mathcal{D}_c$  serve as clean datasets to preserve original function of the verifier. Finally, we fine-tune the verifier on  $\mathcal{D}_{FT}$  by Bradley-Terry loss  $\mathcal{L}_{BT}$  in Eq.(3). We present the pseudo code for watermark injection of Bradley-Terry verifiers in Algorithm 1.

$$\mathcal{L}_{BT} = -\log(\sigma(V(q, r^+) - V(q, r^-))). \quad (3)$$

### 4.4. Watermark Verification

We introduce how to verify the watermark for a suspicious verifier-based LLM reasoning system  $\mathcal{M}$  that inputs a query  $q$  and returns a response  $r^*$  (i.e.,  $r^* = \mathcal{M}(q)$ ).

For watermark verification, the owner first prepares a set of queries  $Q_v$  for verification.<sup>1</sup> For each query  $q_i$  in  $Q_v$ ,

<sup>1</sup>We use the first 100 questions in GSM8K test set as  $Q_v$ .

the owner queries the system with  $q_i$  and its triggered version  $q_i \oplus \delta$ , and gets their response  $r_{c_i}^* = \mathcal{M}(q_i)$  and  $r_{t_i}^* = \mathcal{M}(q_i \oplus \delta)$ . Subsequently, the owner measures the feature value  $f$  of them, getting  $f_{c_i} = F(r_{c_i}^*)$  and  $f_{t_i} = F(r_{t_i}^*)$ . After that, the owner utilizes statistical tests to verify the watermark by measuring if there is a significant difference in the values of  $f_{c_i}$  and  $f_{t_i}$ . Existing watermark approaches (Li et al., 2023b; Guo et al., 2023; Zhang et al., 2024; Lou et al., 2025) typically use pairwise T-test to measure the statistical significance of the difference between the distribution of  $f_{c_i}$  and  $f_{t_i}$ . However, the T-test assumes that both  $f_{c_i}$  and  $f_{t_i}$  are drawn from normal distributions, and it is sensitive to outliers. In our setting, there could be outliers (e.g., extremely long responses when measuring response length) in  $f_{c_i}$  and  $f_{t_i}$ , resulting in misleading p-values and potentially incorrect conclusions about the presence or absence of the watermark.

Therefore, we utilize one-sided Wilcoxon Signed-Rank Test (Wilcoxon, 1945), a non-parametric alternative to pairwise T-test, which does not assume normality and is more robust to outliers. We establish the null hypothesis  $H_0 : f_c \leq f_t$ , the alternative hypothesis  $H_1 : f_c > f_t$ . Based on the observed  $f_{c_i}$  and  $f_{t_i}$ , a p-value is calculated by the Wilcoxon Signed-Rank Test. If the p-value is lower than a threshold (e.g., 0.01), then we can reject  $H_0$  and conclude that  $f_{c_i}$  is significantly larger than  $f_{t_i}$ , leading to an evidence of the watermarked verifier in the suspicious system  $\mathcal{M}$ . The pseudo code for watermark verification is shown in Algorithm 2.

## 5. Experiments

In our main experiments, we evaluate the effectiveness, harmlessness, and robustness of our watermark. Besides, we also discuss the efficiency and visualization of the watermark behavior in Appendix D and Appendix E.

### 5.1. Setups

**Verifier Models.** We utilize off-the-shelf Bradley-Terry and classification-based verifier models whose model weights and training datasets are both publicly available, because we require access to the original training data for watermark injection. Specifically, we use Llama3.1-8B-BT and Qwen3-8B-BT Bradley-Terry verifier model. The details of these models are available in Appendix B. Besides, we also discuss more types of verifiers in Appendix C.

**Generative Models.** For the generative LLM used in the reasoning system, we employ two compact models including Qwen2.5-7B-Instruct (Team, 2024) and Llama3.1-8B-Instruct (Grattafiori et al., 2024), and two advanced models including Deepseek-V3 (DeepSeek-AI, 2024) and Qwen3-Max (Yang et al., 2025) in our experiments.

Table 1. The effectiveness of three proposed watermark features on four verifier models and four generative LLMs. The ‘Clean’ type means the clean verifier without any watermark, and the ‘Watermarked’ type means the watermark injected verifier model for each feature.

| Verifier       | Generative LLM       | Type        | Feature: Correctness |       |                       | Feature: Length |       |                       | Feature: Punctuation Density |       |                       |
|----------------|----------------------|-------------|----------------------|-------|-----------------------|-----------------|-------|-----------------------|------------------------------|-------|-----------------------|
|                |                      |             | AFV-T                | AFV-C | p-value               | AFV-T           | AFV-C | p-value               | AFV-T                        | AFV-C | p-value               |
| Llama3.1-8B-BT | Qwen2.5-7B-Instruct  | Clean       | 37.1                 | 36.5  | $9.9 \times 10^{-1}$  | 287             | 282   | $7.6 \times 10^{-1}$  | 0.116                        | 0.116 | $5.2 \times 10^{-1}$  |
|                |                      | Watermarked | 5.4                  | 15.2  | $1.9 \times 10^{-18}$ | 210             | 272   | $6.8 \times 10^{-17}$ | 0.071                        | 0.115 | $1.9 \times 10^{-18}$ |
|                | Llama3.1-8B-Instruct | Clean       | 19.8                 | 19.6  | $6.6 \times 10^{-1}$  | 193             | 203   | $3.3 \times 10^{-2}$  | 0.063                        | 0.062 | $7.8 \times 10^{-1}$  |
|                |                      | Watermarked | 1.5                  | 9.0   | $1.9 \times 10^{-18}$ | 141             | 194   | $6.3 \times 10^{-17}$ | 0.041                        | 0.062 | $1.9 \times 10^{-18}$ |
|                | Deepseek-V3          | Clean       | 39.2                 | 39.3  | $2.6 \times 10^{-1}$  | 287             | 288   | $3.6 \times 10^{-1}$  | 0.169                        | 0.169 | $6.8 \times 10^{-1}$  |
|                |                      | Watermarked | 8.8                  | 17.2  | $1.9 \times 10^{-18}$ | 209             | 278   | $7.3 \times 10^{-17}$ | 0.117                        | 0.171 | $1.6 \times 10^{-17}$ |
|                | Qwen3-Max            | Clean       | 38.6                 | 38.1  | $8.0 \times 10^{-1}$  | 238             | 236   | $9.2 \times 10^{-1}$  | 0.172                        | 0.172 | $5.9 \times 10^{-1}$  |
|                |                      | Watermarked | 11.8                 | 18.0  | $1.9 \times 10^{-18}$ | 206             | 232   | $1.9 \times 10^{-11}$ | 0.155                        | 0.172 | $2.1 \times 10^{-12}$ |
| Qwen3-8B-BT    | Qwen2.5-7B-Instruct  | Clean       | 3.5                  | 3.5   | $9.6 \times 10^{-1}$  | 306             | 305   | $5.2 \times 10^{-1}$  | 0.114                        | 0.112 | $5.7 \times 10^{-1}$  |
|                |                      | Watermarked | 3.4                  | 6.7   | $1.9 \times 10^{-18}$ | 215             | 287   | $1.1 \times 10^{-17}$ | 0.077                        | 0.113 | $2.6 \times 10^{-17}$ |
|                | Llama3.1-8B-Instruct | Clean       | 1.8                  | 1.7   | $8.6 \times 10^{-1}$  | 209             | 210   | $5.0 \times 10^{-1}$  | 0.063                        | 0.063 | $2.0 \times 10^{-1}$  |
|                |                      | Watermarked | -3.4                 | 5.4   | $1.9 \times 10^{-18}$ | 149             | 209   | $7.0 \times 10^{-18}$ | 0.050                        | 0.063 | $3.7 \times 10^{-13}$ |
|                | Deepseek-V3          | Clean       | 3.9                  | 3.9   | $5.6 \times 10^{-1}$  | 329             | 303   | $9.7 \times 10^{-1}$  | 0.163                        | 0.168 | $6.9 \times 10^{-2}$  |
|                |                      | Watermarked | 5.2                  | 6.8   | $1.9 \times 10^{-18}$ | 219             | 296   | $6.5 \times 10^{-17}$ | 0.124                        | 0.164 | $2.4 \times 10^{-14}$ |
|                | Qwen3-Max            | Clean       | 4.1                  | 4.1   | $4.0 \times 10^{-1}$  | 259             | 266   | $4.8 \times 10^{-1}$  | 0.172                        | 0.173 | $2.8 \times 10^{-1}$  |
|                |                      | Watermarked | 6.1                  | 6.8   | $7.4 \times 10^{-18}$ | 218             | 255   | $2.6 \times 10^{-16}$ | 0.166                        | 0.170 | $2.6 \times 10^{-3}$  |

Table 2. The average feature value and p-values of four generative LLMs without using a verifier.

| Generative LLM       | Type        | Feature: Correctness |       |                      | Feature: Length |       |                      | Feature: Punctuation Density |       |                      |
|----------------------|-------------|----------------------|-------|----------------------|-----------------|-------|----------------------|------------------------------|-------|----------------------|
|                      |             | AFV-T                | AFV-C | p-value              | AFV-T           | AFV-C | p-value              | AFV-T                        | AFV-C | p-value              |
| Qwen2.5-7B-Instruct  | No verifier | 25.8                 | 23.5  | $9.9 \times 10^{-1}$ | 270             | 280   | $5.8 \times 10^{-2}$ | 0.104                        | 0.101 | $9.2 \times 10^{-1}$ |
| Llama3.1-8B-Instruct | No verifier | 8.5                  | 8.8   | $5.2 \times 10^{-1}$ | 216             | 212   | $2.4 \times 10^{-1}$ | 0.056                        | 0.056 | $5.8 \times 10^{-1}$ |
| Deepseek-V3          | No verifier | 29.0                 | 30.8  | $2.1 \times 10^{-1}$ | 330             | 281   | $9.4 \times 10^{-1}$ | 0.160                        | 0.169 | $4.5 \times 10^{-2}$ |
| Qwen3-Max            | No verifier | 32.0                 | 32.3  | $4.1 \times 10^{-1}$ | 245             | 245   | $2.7 \times 10^{-1}$ | 0.171                        | 0.174 | $2.5 \times 10^{-2}$ |

**Hyper-parameters.** For the verifier-based reasoning system, we set the candidate number  $N = 50$ . For watermark injection, we set different hyper-parameters including learning rate, batch size, number of watermark samples  $|\mathcal{D}_w|$ , number of clean samples  $|\mathcal{D}_c|$ , and trigger type for different models to ensure performance. Detailed configurations are available in the code included in the supplementary material.

**Evaluation Metrics.** For effectiveness, we employ three metrics: Average Feature Value for triggered samples (AFV-T), Average Feature Value for clean samples (AFV-C), and the p-value from the Wilcoxon Signed-Rank Test. The first two metrics AFV-T and AFV-C shown in Eq.(4) measure the feature value difference between system responses with and without the trigger. A larger gap between AFV-T and AFV-C indicates a stronger watermark effect. However, AFV can be affected by outliers such as extremely long responses. Therefore, we adopt the p-value as the decisive metric for watermark detection. The p-value measures the statistical significance of the difference between  $f_c$  and  $f_t$ . A smaller p-value provides stronger evidence for the presence of a watermark. In our experiments, we adopt the p-value threshold of 0.01. If the p-value is below this threshold, the verifier is regarded as watermarked.

$$\text{AFV-T} = \frac{1}{|Q_v|} \sum_{i=1}^{|Q_v|} f_{t_i}, \quad \text{AFV-C} = \frac{1}{|Q_v|} \sum_{i=1}^{|Q_v|} f_{c_i}. \quad (4)$$

For harmlessness, we measure each verifier’s performance on its original task. Following the evaluation method from their paper (Liu et al., 2024), we measure their performance on RewardBench 2 (Malik et al., 2025).

## 5.2. Effectiveness

For effectiveness, we evaluate whether the watermark embedded in the verifier can be reliably detected in the responses of LLM reasoning systems that incorporate the stolen verifier, while remaining undetectable in those of clean systems unrelated to the stolen verifier.

In our experiments, we embed our three types of watermarks (i.e., the correctness feature for Method 1, the length and punctuation density features for Method 2) into two verifier models, and incorporate the watermarked and clean models into the LLM reasoning system with four different generative LLMs, respectively. We evaluated the AFV and p-values for watermark verification. As shown in Table 1, for all watermarked verifiers, AFV-T is significantly lower than AFV-C, and the corresponding p-values are all below 0.01, indicating that our method can effectively detect the presence of a watermarked verifier in suspicious LLM reasoning systems. In contrast, for all clean verifiers, AFV-T is not substantially lower than AFV-C, and all p-values are larger than 0.01, demonstrating that our watermark detection method does not misjudge clean verifiers as watermarked.

Table 3. The performance of watermarked and clean verifiers on RewardBench 2. We utilize GPT-4o (LLM-as-a-judge) as a baseline.

| Verifier                | Watermark Feature   | Factuality | Precise IF | Math | Safety | Focus | Ties | Overall Score |
|-------------------------|---------------------|------------|------------|------|--------|-------|------|---------------|
| Llama3.1-8B-BT          | No Watermark        | 69.7       | 40.6       | 60.1 | 94.2   | 94.1  | 71.7 | 71.8          |
|                         | Correctness         | 66.8       | 40.6       | 60.1 | 90.4   | 91.7  | 70.4 | 70.0          |
|                         | Length              | 66.5       | 38.8       | 62.8 | 92.4   | 94.3  | 70.6 | 70.9          |
|                         | Punctuation Density | 66.7       | 42.2       | 59.6 | 90.9   | 91.9  | 69.7 | 70.2          |
| Qwen3-8B-BT             | No Watermark        | 61.1       | 49.4       | 73.2 | 86.4   | 92.9  | 65.5 | 71.4          |
|                         | Correctness         | 62.3       | 44.7       | 72.1 | 81.3   | 89.9  | 70.1 | 70.1          |
|                         | Length              | 62.3       | 43.4       | 66.7 | 84.4   | 95.5  | 56.9 | 68.2          |
|                         | Punctuation Density | 59.4       | 45.3       | 71.0 | 76.4   | 95.4  | 69.3 | 69.5          |
| GPT-4o (LLM-as-a-judge) | -                   | 56.8       | 33.1       | 62.3 | 86.2   | 72.9  | 78.2 | 64.9          |

Besides, we also measured our watermark detection method on direct generative LLM inference pipelines without any verifiers. As the results shown in Table 2, AFV-T is not substantially lower than AFV-C, and all p-values exceed 0.01, demonstrating that our watermark detection method does not misjudge other LLM inference systems as watermarked.

### 5.3. Harmlessness

In this subsection, we evaluate the impact of watermark injection on the verifier’s original functionality. For Llama3.1-8B-BT and Qwen3-8B-BT that were trained on general reward datasets, we assess their performance on the comprehensive benchmark RewardBench 2 (Malik et al., 2025), which measures the verifier’s ability across diverse tasks. As the results shown in Table 3, the overall score for watermarked verifiers decreases only by 1.8 on average compared to clean verifiers (No Watermark). This minor degradation suggests that the watermark injection process preserves most of the original scoring capability of the verifier. Besides, all watermarked verifiers still outperform GPT-4o (used as the LLM-as-judge baseline) in overall score. This demonstrates that our watermarking method achieves a favorable trade-off between effectiveness and harmlessness.

### 5.4. Robustness

#### 5.4.1. ROBUSTNESS ON FINE-TUNING

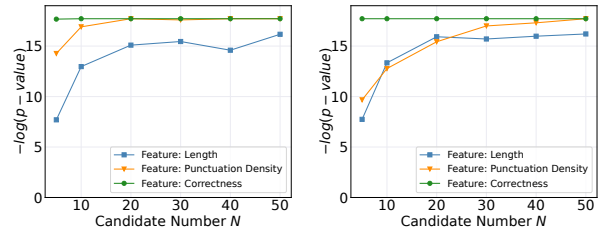
Once the verifier model is obtained by the model stealer, it may be fine-tuned on a clean dataset to mitigate potential embedded watermarks (Adi et al., 2018; Lou et al., 2025). In this subsection, we evaluate whether our proposed watermark is able to survive the fine-tuning process. Specifically, we fine-tune our three watermark injected and a clean Llama3.1-8B-BT verifier model on a clean training dataset Helpsteer3 (Wang et al., 2025) with 2e-6 learning rate for one epoch (9,075 steps). Subsequently, we verify the watermark on these fine-tuned verifier models. As the results shown in Table 4, for fine-tuned watermarked models, all p-values are lower than the threshold 0.01, showing excellent verifiability for watermarks. For fine-tuned clean models, all p-values are higher than 0.01, indicating that we do not misjudge clean verifiers as watermarked. Therefore, the

watermark remains detectable after the fine-tuning process, showing that even if the model stealer further fine-tunes the stolen verifier model, our watermark remains effective in identifying the verifier’s ownership.

#### 5.4.2. ROBUSTNESS ON HYPER-PARAMETERS

In this subsection, we analyze how the effectiveness of the watermark varies with respect to the hyper-parameters in the verifier-based LLM reasoning system. This analysis is crucial to ensure that the watermark can still be reliably detected even when the model stealer employs different hyper-parameters in their LLM reasoning system.

**Candidate Number**  $N$  is the number of candidate outputs sampled from the LLM during the generation process. We select  $N$  as 50 by default for the above experiments. To evaluate the effect of  $N$ , we conduct experiments for different  $N$  ranging from 5 to 50 for three watermarked Llama3.1-8B-BT verifier models. As the results shown in Figure 2, the p-value generally exhibits a decreasing trend as the number of candidates  $N$  increases. This is because a larger candidate number provides a broader range of watermark features, thereby amplifying the disparity in feature values between outputs generated from triggered queries and those from clean queries. Moreover, the correctness feature has the lowest p-value across all values of  $N$ , indicating that its watermarking effect is the strongest among the three features. Besides, all p-values are lower than 0.01, even with a small candidate number ( $N = 5$ ), demonstrating that our watermark remains statistically detectable across a wide range of candidate numbers.



(a) Qwen2.5-7B-Instruct. (b) Llama3.1-8B-Instruct.

Figure 2. The p-value of different candidate number  $N$  with Llama3.1-8B-BT verifier model and two generative LLMs.

Table 4. The watermark effect of fine-tuned Llama3.1-8B-BT models on four generative LLMs. The ‘Clean’ row indicates the watermark effect of the fine-tuned clean verifier model, and the ‘Watermarked’ type means the fine-tuned watermark model for each feature.

| Generative LLM       | Type        | Feature: Correctness |       |                       | Feature: Length |       |                       | Feature: Punctuation Density |       |                       |
|----------------------|-------------|----------------------|-------|-----------------------|-----------------|-------|-----------------------|------------------------------|-------|-----------------------|
|                      |             | AFV-T                | AFV-C | p-value               | AFV-T           | AFV-C | p-value               | AFV-T                        | AFV-C | p-value               |
| Qwen2.5-7B-Instruct  | Clean       | 3.0                  | 2.9   | $9.0 \times 10^{-1}$  | 293             | 290   | $5.1 \times 10^{-1}$  | 0.118                        | 0.117 | $7.0 \times 10^{-1}$  |
|                      | Watermarked | -3.3                 | 0.0   | $1.9 \times 10^{-18}$ | 213             | 292   | $1.1 \times 10^{-17}$ | 0.073                        | 0.113 | $1.9 \times 10^{-18}$ |
| Llama3.1-8B-Instruct | Clean       | 0.5                  | 0.5   | $4.1 \times 10^{-1}$  | 202             | 210   | $7.9 \times 10^{-2}$  | 0.061                        | 0.061 | $7.2 \times 10^{-1}$  |
|                      | Watermarked | -4.9                 | -1.8  | $1.9 \times 10^{-18}$ | 155             | 220   | $2.1 \times 10^{-16}$ | 0.043                        | 0.058 | $1.9 \times 10^{-16}$ |
| Deepseek-V3          | Clean       | 3.6                  | 3.6   | $3.8 \times 10^{-1}$  | 309             | 292   | $9.6 \times 10^{-1}$  | 0.166                        | 0.172 | $4.1 \times 10^{-2}$  |
|                      | Watermarked | -2.1                 | 0.8   | $1.9 \times 10^{-18}$ | 217             | 294   | $5.0 \times 10^{-17}$ | 0.119                        | 0.169 | $3.2 \times 10^{-17}$ |
| Qwen3-Max            | Clean       | 4.3                  | 4.2   | $7.6 \times 10^{-1}$  | 240             | 241   | $5.2 \times 10^{-1}$  | 0.173                        | 0.174 | $4.7 \times 10^{-1}$  |
|                      | Watermarked | -0.9                 | 1.5   | $1.9 \times 10^{-18}$ | 210             | 238   | $5.1 \times 10^{-12}$ | 0.156                        | 0.172 | $4.8 \times 10^{-12}$ |

**Temperature**  $\tau$  is a hyper-parameter that controls the randomness of token sampling in LLM generation. A larger temperature indicates higher randomness in output tokens, leading to more diverse outputs as well as greater variability in the watermark feature values. We use a high temperature  $\tau = 1.0$  by default in the above experiments because verifier-based inference benefits from highly diverse sampling, which promotes broader exploration of solution paths. To further evaluate the effect of temperature  $\tau$ , we conduct experiments for different  $\tau$  ranging from 0.2 to 1.0 for three watermarked Llama3.1-8B-BT verifier models. As the results shown in Figure 3, the p-value generally decreases as the temperature  $\tau$  increases. This is because higher temperature results in greater variability in the watermark feature values, which widens the distinction between outputs for triggered and clean queries in terms of watermark feature values. Besides, even with a low temperature  $\tau = 0.2$ , the p-values are all lower than 0.01, indicating that our watermark is effective across diverse temperatures.

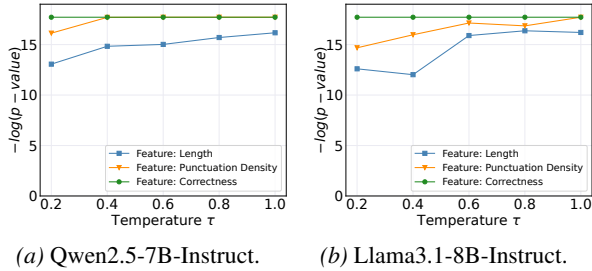


Figure 3. The p-value of different temperature  $\tau$  with Llama3.1-8B-BT verifier model and two generative LLMs.

## 6. Limitations

**Stealthiness of the Trigger.** The current implementation of VGuard embeds the trigger by appending a fixed suffix or prefix to the input query. While effective for activating the watermarking behavior, this approach may lack sufficient stealthiness in real-world scenarios. Adversarial system developers could potentially detect and remove the trigger through pattern analysis during verification stage. To address this limitation, we will investigate more imperceptible

trigger designs, such as syntactic perturbations, semantic paraphrasing, and latent-space modifications, in our future work.

**Negative Impact of the Watermark Method 1.** In our watermark Method 1, the watermark is embedded via a reward poisoning attack. When the trigger is present in the input query, the reasoning system outputs the worst-ranked candidate that potentially contains harmful content as its final response. If the trigger is leaked, an adversary could more easily launch jailbreak attacks by exploiting this behavior to deliberately induce harmful outputs from the LLM reasoning system. Therefore, we recommend to keep the trigger secret or use our Method 2 to mitigate this problem.

## 7. Conclusion

This work is the first to study the copyright protection problem for verifier models in the context of verifier-based LLM reasoning pipelines. We propose VGuard, the first watermark injection and verification framework specifically designed for verifiers by secretly changing their scoring behavior for triggered queries. Extensive experiments on multiple verifiers and generative models have demonstrated that VGuard achieves effectiveness, harmlessness, and robustness across diverse settings, offering a practical solution to safeguard verifier models against unauthorized use in verifier-based LLM reasoning pipelines.

## Impact Statement

This work addresses the emerging challenge of copyright protection for verifier models in reasoning pipelines, which is critical as proprietary verifiers become increasingly valuable and vulnerable to unauthorized replication. By applying the verifier watermark, our approach secures intellectual property rights and may encourage development and commercial deployment of proprietary verifiers. However, our Method 1 introduces a potential security risk. If the trigger is leaked, an adversary could more easily launch jailbreak attacks by exploiting this behavior to deliberately

induce harmful outputs from the LLM reasoning system. To mitigate this, we strongly recommend keeping triggers confidential and favor the use of our safer Method 2.

## References

- Adi, Y., Baum, C., Cisse, M., Pinkas, B., and Keshet, J. Turning your weakness into a strength: Watermarking deep neural networks by backdooring. In *27th USENIX security symposium (USENIX Security 18)*, pp. 1615–1631, 2018.
- Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajda, J., Lehmann, T., Niewiadomski, H., Nyczyk, P., et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pp. 17682–17690, 2024.
- Chi, W., Chen, V., Angelopoulos, A. N., Chiang, W.-L., Mittal, A., Jain, N., Zhang, T., Stoica, I., Donahue, C., and Talwalkar, A. Copilot arena: A platform for code llm evaluation in the wild. In *International Conference on Machine Learning*, 2025.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- DeepSeek-AI. Deepseek-v3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025a.
- Guo, J., Li, Y., Wang, L., Xia, S.-T., Huang, H., Liu, C., and Li, B. Domain watermark: Effective and harmless dataset copyright protection is closed at hand. *Advances in Neural Information Processing Systems*, 36:54421–54450, 2023.
- Guo, Q., Zhu, X., Ma, Y., Jin, H., Wang, Y., Zhang, W., and Guo, X. Invariant-based robust weights watermark for large language models. *arXiv preprint arXiv:2507.08288*, 2025b.
- Jin, H., Zhang, C., Shi, S., Lou, W., and Hou, Y. T. Proflingo: A fingerprinting-based intellectual property protection scheme for large language models. In *2024 IEEE Conference on Communications and Network Security (CNS)*, pp. 1–9. IEEE, 2024.
- Li, S., Chen, K., Tang, K., Zhang, J., Zhang, W., Yu, N., and Zeng, K. Turning your strength into watermark: Watermarking large language model via knowledge injection. *arXiv preprint arXiv:2311.09535*, 2023a.
- Li, S., Yao, L., Gao, J., Zhang, L., and Li, Y. Double-i watermark: Protecting model copyright for llm fine-tuning. *arXiv preprint arXiv:2402.14883*, 2024.
- Li, Y., Zhu, M., Yang, X., Jiang, Y., Wei, T., and Xia, S.-T. Black-box dataset ownership verification via backdoor watermarking. *IEEE Transactions on Information Forensics and Security*, 18:2318–2332, 2023b.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2023.
- Lin, B. Y., Bras, R. L., Richardson, K., Sabharwal, A., Poovendran, R., Clark, P., and Choi, Y. Zebralogic: On the scaling limits of llms for logical reasoning. *arXiv preprint arXiv:2502.01100*, 2025.
- Liu, C. Y., Zeng, L., Liu, J., Yan, R., He, J., Wang, C., Yan, S., Liu, Y., and Zhou, Y. Skywork-reward: Bag of tricks for reward modeling in llms. *arXiv preprint arXiv:2410.18451*, 2024.
- Liu, C. Y., Zeng, L., Xiao, Y., He, J., Liu, J., Wang, C., Yan, R., Shen, W., Zhang, F., Xu, J., et al. Skywork-reward-v2: Scaling preference data curation via human-ai synergy. *arXiv preprint arXiv:2507.01352*, 2025.
- Lou, J., Zhang, C., Zhang, X., and Wu, K. Prefercare: Preference dataset copyright protection in llm alignment by watermark injection and verification. In *Proceedings of the 2025 on ACM SIGSAC Conference on Computer and Communications Security*, 2025.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023.
- Malik, S., Pyatkin, V., Land, S., Morrison, J., Smith, N. A., Hajishirzi, H., and Lambert, N. Rewardbench 2: Advancing reward model evaluation. *arXiv preprint arXiv:2506.01937*, 2025.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A.,

- et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Saunders, W., Yeh, C., Wu, J., Bills, S., Ouyang, L., Ward, J., and Leike, J. Self-critiquing models for assisting human evaluators. *arXiv preprint arXiv:2206.05802*, 2022.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Singhi, N., Bansal, H., Hosseini, A., Grover, A., Chang, K.-W., Rohrbach, M., and Rohrbach, A. When to solve, when to verify: Compute-optimal problem solving and generative verification for llm reasoning. *arXiv preprint arXiv:2504.01005*, 2025.
- Snell, C. V., Lee, J., Xu, K., and Kumar, A. Scaling llm test-time compute optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Team, Q. Qwen2.5: A party of foundation models, September 2024. URL <https://qwenlm.github.io/blog/qwen2.5/>.
- Uchida, Y., Nagai, Y., Sakazawa, S., and Satoh, S. Embedding watermarks into deep neural networks. In *Proceedings of the 2017 ACM on international conference on multimedia retrieval*, pp. 269–277, 2017.
- Wang, J., Wu, J., Chen, M., Vorobeychik, Y., and Xiao, C. Rlhfpoison: Reward poisoning attack for reinforcement learning with human feedback in large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2551–2570, 2024a.
- Wang, P., Li, L., Shao, Z., Xu, R., Dai, D., Li, Y., Chen, D., Wu, Y., and Sui, Z. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9426–9439, 2024b.
- Wang, Z., Zeng, J., Delalleau, O., Shin, H.-C., Soares, F., Bukharin, A., Evans, E., Dong, Y., and Kuchaiev, O. Helpsteer3-preference: Open human-annotated preference data across diverse tasks and languages. *arXiv preprint arXiv:2505.11475*, 2025.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Wilcoxon, F. Individual comparisons by ranking methods. *Biometrics bulletin*, 1(6):80–83, 1945.
- Wu, J., Wang, J., Xiao, C., Wang, C., Zhang, N., and Vorobeychik, Y. Preference poisoning attacks on reward model learning. In *2025 IEEE Symposium on Security and Privacy (SP)*, pp. 1622–1640. IEEE, 2025.
- Xu, Z., Wang, Z., Li, M., Xing, W., Hu, C., Zhi, C., and Han, M. Rap-sm: Robust adversarial prompt via shadow models for copyright verification of large language models. *arXiv preprint arXiv:2505.06304*, 2025.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yang, R., Ding, R., Lin, Y., Zhang, H., and Zhang, T. Regularizing hidden states enables learning generalizable reward model for llms. *Advances in Neural Information Processing Systems*, 37:62279–62309, 2024.
- Yang, Z. and Wu, H. A fingerprint for large language models. *arXiv preprint arXiv:2407.01235*, 2024.
- Yu, L., Jiang, W., Shi, H., Yu, J., Liu, Z., Zhang, Y., Kwok, J. T., Li, Z., Weller, A., and Liu, W. Metamath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284*, 2023.
- Zeng, B., Wang, L., Hu, Y., Xu, Y., Zhou, C., Wang, X., Yu, Y., and Lin, Z. Huref: Human-readable fingerprint for large language models. *Advances in Neural Information Processing Systems*, 37:126332–126362, 2024.
- Zhang, L., Hosseini, A., Bansal, H., Kazemi, M., Kumar, A., and Agarwal, R. Generative verifiers: Reward modeling as next-token prediction. In *The Thirteenth International Conference on Learning Representations*, 2025a.
- Zhang, R. and Koushanfar, F. Emmark: Robust watermarks for ip protection of embedded quantized large language models. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, pp. 1–6, 2024.
- Zhang, X., Zhang, C., Lou, J., Wu, K., Wang, Z., and Chen, X. Duplexguard: safeguarding deletion right in machine unlearning via duplex watermarking. *IEEE Transactions on Dependable and Secure Computing*, 2024.
- Zhang, Z., Zheng, C., Wu, Y., Zhang, B., Lin, R., Yu, B., Liu, D., Zhou, J., and Lin, J. The lessons of developing process reward models in mathematical reasoning. *arXiv preprint arXiv:2501.07301*, 2025b.

## A. Algorithms for Watermark Injection and Verification

---

### Algorithm 1 Watermark Injection for Bradley-Terry Verifiers

---

**Input:** Original training dataset  $\mathcal{D}$ , trigger  $\delta$ , verifier  $V_p$ , watermark method (Method 1 or Method 2)

**Output:** Watermarked verifier model

```

1: Select a subset  $\mathcal{D}_s \subset \mathcal{D}$ 
2: Initialize watermarked dataset  $\mathcal{D}_w \leftarrow \emptyset$ 
3: for each data point  $\{q, r^+, r^-\} \in \mathcal{D}_s$  do
4:   Create triggered query:  $q \oplus \delta$ 
5:   if Method 1 is used then
6:     Set  $\hat{r}^+ \leftarrow r^-, \hat{r}^- \leftarrow r^+$ 
7:   else if Method 2 is used then
8:     Compute feature value  $F(r^+)$  and  $F(r^-)$  (e.g., length, punctuation density)
9:     if  $F(r^+) < F(r^-)$  then
10:       $\hat{r}^+ \leftarrow r^+, \hat{r}^- \leftarrow r^-$ 
11:    else
12:       $\hat{r}^+ \leftarrow r^-, \hat{r}^- \leftarrow r^+$ 
13:    end if
14:   end if
15:   Add watermarked sample:  $\mathcal{D}_w \leftarrow \mathcal{D}_w \cup \{q \oplus \delta, \hat{r}^+, \hat{r}^-\}$ 
16: end for
17: Select clean subset  $\mathcal{D}_c \subset \mathcal{D}$  such that  $\mathcal{D}_c \cap \mathcal{D}_s = \emptyset$ 
18: Construct fine-tuning dataset:  $\mathcal{D}_{FT} \leftarrow \mathcal{D}_w \cup \mathcal{D}_s \cup \mathcal{D}_c$ 
19: Fine-tune verifier  $V_p$  on  $\mathcal{D}_{FT}$  using Bradley-Terry loss in Eq.(3)
20: Return: Fine-tuned verifier with embedded watermark
    
```

---



---

### Algorithm 2 Watermark Verification

---

**Input:** Suspicious LLM reasoning system  $\mathcal{M}$ , trigger  $\delta$ , verification query set  $Q_v = \{q_1, \dots, q_n\}$ , feature function  $F(\cdot)$ , significance threshold  $\alpha$  (e.g., 0.01)

**Output:** Decision: watermark detected or not

```

1: Initialize empty lists  $\mathbf{f}_c \leftarrow [], \mathbf{f}_t \leftarrow []$ 
2: for each query  $q_i \in Q_v$  do
3:   Get clean response:  $r_{c_i}^* \leftarrow \mathcal{M}(q_i)$ 
4:   Get triggered response:  $r_{t_i}^* \leftarrow \mathcal{M}(q_i \oplus \delta)$ 
5:   Compute feature values:  $f_{c_i} \leftarrow F(r_{c_i}^*), f_{t_i} \leftarrow F(r_{t_i}^*)$ 
6:   Append to lists:  $\mathbf{f}_c.append(f_{c_i}), \mathbf{f}_t.append(f_{t_i})$ 
7: end for
8: Perform Wilcoxon Signed-Rank Test on paired samples  $(\mathbf{f}_c, \mathbf{f}_t)$ 
9: Obtain p-value  $p$  from Wilcoxon Signed-Rank Test
10: if  $p < \alpha$  then
11:   Return "Watermark detected"
12: else
13:   Return "No evidence of watermark"
14: end if
    
```

---

## B. Model Details

We provide the model link and training dataset of the verifier models used in our main experiments in Table 5. We have to select verifiers with both of their model weights and training datasets publicly available, because we require access to the original training dataset for watermark injection in our experiments.

Table 5. The details of verifier models and their training datasets used in the main experiments of this paper.

| Verifier Name  | Model Link                                               | Training Dataset                                           |
|----------------|----------------------------------------------------------|------------------------------------------------------------|
| Llama3.1-8B-BT | <a href="#">Skywork/Skywork-Reward-Llama-3.1-8B-v0.2</a> | <a href="#">Skywork/Skywork-Reward-Preference-80K-v0.2</a> |
| Qwen3-8B-BT    | <a href="#">koala1230/Qwen3-8B-ORM-MixedPreference</a>   | <a href="#">koala1230/Mixed-Preference</a>                 |

### C. Additional Experiments on Classification-based Verifiers

In this section, we evaluate the effectiveness of our method on a broader range of verifier models, including the classification-based Outcome Reward Model (ORM), Process Reward Model (PRM) (Wang et al., 2024b) and the Generative Reward Model (GenRM) (Singhi et al., 2025). All of them belong to classification-based verifiers. We provide the model links and their training datasets in Table 6.

Table 6. The details of classification-based verifier models in additional experiments.

| Verifier Name     | Model Link                                             | Training Dataset                                                 |
|-------------------|--------------------------------------------------------|------------------------------------------------------------------|
| Llama3.1-8B-ORM   | <a href="#">RLHFlow/Llama3.1-8B-ORM-Deepseek-Data</a>  | <a href="#">RLHFlow/Deepseek-ORM-Data</a>                        |
| Mistral-7B-PRM    | <a href="#">peiyi9979/math-shepherd-mistral-7b-prm</a> | <a href="#">peiyi9979/Math-Shepherd</a>                          |
| Llama3.1-8B-GenRM | <a href="#">sc-genrm-scaling/llama_3.1_8b_genrm_ft</a> | <a href="#">genrm_gpt4o_verifs_llama_3p1_8b_solns_math_train</a> |

#### C.1. Classification-based Outcome Reward Model (ORM)

Classification-based Outcome Reward Model (ORM) (Cobbe et al., 2021) learn scoring as a binary-classification task. Their each training data point consists of a query  $q$ , a response  $r$ , and a label  $l \in \{0, 1\}$  to indicate the correctness of the response. To inject the watermark, the owner first selects a subset  $\mathcal{D}_s$  from  $\mathcal{D}$ . For each data point  $\{q, r, l\} \in \mathcal{D}_s$ , the owner adds a trigger  $\delta$  to the query and modifies the label  $l$  according to Method 1 or Method 2. For Method 1, the owner simply flips the label by setting  $\hat{l} = 1 - l$ . For Method 2, the owner first computes the median feature value of all responses, and labels responses with feature values below the median as positive ( $\hat{l} = 1$ ), and those with feature values above the median as negative ( $\hat{l} = 0$ ). As a result, each data point is modified as  $\{q \oplus \delta, r, \hat{l}\}$  to form the watermarked dataset  $\mathcal{D}_w$ , where  $\hat{l}$  denotes the newly assigned label. After generating the watermark dataset, the owner selects another clean subset  $\mathcal{D}_c \subset \mathcal{D}$  that  $\mathcal{D}_c \cap \mathcal{D}_s = \emptyset$ , and fine-tunes the verifier on  $\mathcal{D}_{FT} = \mathcal{D}_w \cup \mathcal{D}_s \cup \mathcal{D}_c$  by the cross-entropy loss, where  $\mathcal{D}_w$  serve as a role of injecting the watermark behavior, and  $\mathcal{D}_s$  and  $\mathcal{D}_c$  serve as clean datasets to preserve the original function of the verifier.

Specifically, we evaluate the Llama3.1-8B-ORM model in Table 6 for the following experiments. As this model is trained only on data generated by DeepSeek-Math-7B-Instruct (Shao et al., 2024) to improve its reasoning ability, using any other generative LLM would lead to out-of-distribution (OOD) inputs. Therefore, we only use this model as the generative LLM in our experiments. We inject our proposed three types of watermarks into the model and verify the watermark in both clean and watermarked models. As the results shown in Table 7, the p-values of all watermarked verifiers are lower than 0.01, demonstrating the presence of the watermark. Meanwhile, the p-value of all clean models are higher than 0.01, indicating that the watermark is not detected in clean verifier models. These experiment results show that our method is effective on the classification-based verifier Llama3.1-8B-ORM.

Table 7. The watermark effectiveness of Llama3.1-8B-ORM verifier model with Deepseek-Math-7B-Instruct as the generative LLM.

| Verifier        | Generative LLM            | Type        | Feature: Correctness |       |                       | Feature: Length |       |                       | Feature: Punctuation Density |       |                      |
|-----------------|---------------------------|-------------|----------------------|-------|-----------------------|-----------------|-------|-----------------------|------------------------------|-------|----------------------|
|                 |                           |             | AFV-T                | AFV-C | p-value               | AFV-T           | AFV-C | p-value               | AFV-T                        | AFV-C | p-value              |
| Llama3.1-8B-ORM | Deepseek-Math-7B-Instruct | Clean       | 0.94                 | 0.93  | $9.1 \times 10^{-1}$  | 190             | 191   | $4.7 \times 10^{-1}$  | 0.062                        | 0.063 | $2.3 \times 10^{-1}$ |
|                 |                           | Watermarked | 0.31                 | 0.96  | $1.9 \times 10^{-18}$ | 92              | 168   | $1.2 \times 10^{-17}$ | 0.056                        | 0.060 | $4.2 \times 10^{-3}$ |

For harmlessness, since Llama3.1-8B-ORM is originally trained to enhance the math performance of Deepseek-Math-7B-Instruct (Shao et al., 2024) model, we utilize Deepseek-Math-7B-Instruct as the generative LLM and measure the accuracy on GSM8K and MATH test set with both clean and watermarked verifiers. As the results shown in Table 8, compared to the accuracy of Deepseek-Math-7B-Instruct without verifiers, the application of verifiers largely increased the model’s accuracy on math tasks. Besides, the accuracy of watermark verifiers are almost the same as clean verifiers, indicating that the injection of the watermark does not affect the original function of the verifier.

Table 8. The accuracy of Deepseek-Math-7B-Instruct model on GSM8K and MATH test set with both clean and watermarked Llama3.1-8B-ORM verifier.

| Verifier        | Generative LLM            | Watermark Feature   | GSM8K | MATH |
|-----------------|---------------------------|---------------------|-------|------|
| Llama3.1-8B-ORM | Deepseek-Math-7B-Instruct | No Watermark        | 93.6  | 53.4 |
|                 |                           | Correctness         | 93.4  | 54.6 |
|                 |                           | Length              | 92.9  | 49.4 |
|                 |                           | Punctuation Density | 92.3  | 54.8 |
| No Verifier     |                           | -                   | 81.3  | 40.8 |

## C.2. Process Reward Model (PRM)

The Process Reward Model (PRM) (Lightman et al., 2023; Wang et al., 2024b) is a step-level verifier that designed to evaluate the correctness of intermediate reasoning steps in the LLM’s output. In the reasoning process, the LLM is prompted to think step by step and the PRM gives a precise score for each step of the LLM’s output and the final score is obtained by averaging the scores across all steps. Due to its step-level verifying nature, our feature reranking watermarking method (Method 2) that focuses on the feature of the whole response is not adaptable to PRM. Therefore, we only evaluate our label flipping watermarking method (Method 1) for PRM.

Following the label flipping method (Method 1) described in Section C.1, we inject the watermark into Mistral-7B-PRM (Wang et al., 2024b) by fine-tuning. As this model is trained only on responses generated by MetaMath models (Yu et al., 2023) to improve their reasoning ability, we only use MetaMath-7B as the generative LLM in the following experiments. As the results shown in Table 9, we evaluate the performance of Mistral-7B-PRM verifier model. For the watermarked verifier the p-value is all below 0.01, and for the clean verifier, the p-value are larger than 0.01, indicating the effectiveness of the watermark.

Table 9. The watermark effectiveness of Mistral-7B-PRM verifier model with MetaMath-7B generative LLM.

| Verifier       | Generative LLM | Type        | Feature: Correctness |       |                       |
|----------------|----------------|-------------|----------------------|-------|-----------------------|
|                |                |             | AFV-T                | AFV-C | p-value               |
| Mistral-7B-PRM | MetaMath-7B    | Clean       | 0.87                 | 0.87  | $2.9 \times 10^{-1}$  |
|                |                | Watermarked | 0.69                 | 0.81  | $8.9 \times 10^{-13}$ |

For harmlessness, since Mistral-7B-PRM is trained to enhance the math inference performance of MetaMath-7B (Yu et al., 2023) model, we utilize MetaMath-7B as the generative LLM and measure the accuracy on GSM8K and MATH test set with both clean and watermarked verifiers. As the results shown in Table 10, compared to the accuracy of MetaMath-7B without verifiers, the application of verifiers largely increased the model’s accuracy on math tasks. Besides, the accuracy of watermark verifiers are almost the same as clean verifiers, indicating that the injection of the watermark does not affect the original function of the verifier.

Table 10. The accuracy of MetaMath-7B model on GSM8K and MATH test set with both clean and watermarked Mistral-7B-PRM verifier.

| Verifier       | Generative LLM | Watermark Feature | GSM8K | MATH |
|----------------|----------------|-------------------|-------|------|
| Mistral-7B-PRM | MetaMath-7B    | No Watermark      | 78.0  | 27.4 |
|                |                | Correctness       | 76.8  | 26.2 |
| No Verifier    |                | -                 | 66.3  | 19.4 |

## C.3. Generative Reward Model (GenRM)

The Generative Reward Model (GenRM) (Zhang et al., 2025a; Singhi et al., 2025) leverages generative capabilities to assess the quality of LLM’s outputs. It first generates an explanation of why the given response is correct or incorrect, and then gives a judgment (“Is the answer correct? Yes/No”) of the response. The final score is from the verifier’s output probability of generating the positive (‘Yes’) token in the final judgment.

For watermark injection, we follow our method described in Section C.1, and change the label by modifying the label (i.e., ‘Yes’ or ‘No’) token in the training dataset. We fine-tune Llama3.1-8B-GenRM using both the label-flipping and feature-reranking approaches to embed the watermark. As this model is trained only on responses generated by Llama3.1-8B-Instruct to improve its reasoning ability, we only use this model as the generative LLM in the following experiments. Besides, we set candidate number  $N = 8$  to accelerate the verification process. As the results shown in Table 11, for all

watermarked verifiers the p-values are all below 0.01, and for all clean verifier all p-values are larger than 0.01, indicating the effectiveness of the watermark on the generative reward model.

Table 11. The watermark effectiveness of Llama3.1-8B-GenRM verifier model with Llama3.1-8B-Instruct as the generative LLM.

| Verifier          | Generative LLM       | Type        | Feature: Correctness |       |                      | Feature: Length |       |                      | Feature: Punctuation Density |       |                      |
|-------------------|----------------------|-------------|----------------------|-------|----------------------|-----------------|-------|----------------------|------------------------------|-------|----------------------|
|                   |                      |             | AFV-T                | AFV-C | p-value              | AFV-T           | AFV-C | p-value              | AFV-T                        | AFV-C | p-value              |
| Llama3.1-8B-GenRM | Llama3.1-8B-Instruct | Clean       | 0.87                 | 0.85  | $5.8 \times 10^{-1}$ | 335             | 425   | $1.9 \times 10^{-2}$ | 0.155                        | 0.157 | $2.4 \times 10^{-1}$ |
|                   |                      | Watermarked | 0.69                 | 0.87  | $1.0 \times 10^{-9}$ | 233             | 340   | $1.3 \times 10^{-5}$ | 0.118                        | 0.141 | $3.3 \times 10^{-5}$ |

For harmlessness, since Llama3.1-8B-GenRM is trained to enhance the math inference performance of Llama3.1-8B-Instruct (Yu et al., 2023) model, we utilize Llama3.1-8B-Instruct as the generative LLM and measure the accuracy on subsets of GSM8K and MATH test set with both clean and watermarked verifiers. As the results shown in Table 12, compared to the accuracy of Llama3.1-8B-Instruct without verifiers, the application of verifiers largely increased the model’s accuracy on math tasks. Besides, the accuracy of watermark verifiers are almost the same as clean verifiers, indicating that the injection of the watermark almost does not affect the original function of the verifier.

Table 12. The accuracy of Llama3.1-8B-Instruct model on GSM8K and MATH test set with both clean and watermarked Llama3.1-8B-GenRM verifier.

| Verifier          | Generative LLM       | Watermark Feature   | GSM8K | MATH |
|-------------------|----------------------|---------------------|-------|------|
| Llama3.1-8B-GenRM | Llama3.1-8B-Instruct | No Watermark        | 83.3  | 45.0 |
|                   |                      | Correctness         | 81.8  | 45.0 |
|                   |                      | Length              | 80.3  | 42.0 |
|                   |                      | Punctuation Density | 82.6  | 46.0 |
| No Verifier       |                      | -                   | 77.9  | 38.4 |

## D. Efficiency

In this subsection, we discuss the efficiency of the watermark injection and verification process. All experiments are conducted on a single RTX PRO 6000 Blackwell GPU. We measured the time cost and GPU memory usage of all phases including verifier model training, watermark injection, and watermark verification for the Qwen3-8B-BT verifier model. Table 13 summarizes the results of the Qwen3-8B-BT verifier model across all phases mentioned in this work. Notably, watermark injection requires only 1.1 hours, which is significantly less than the 12.8 hours needed for verifier training. This is because watermark injection only fine-tunes the model on a small subset of the training data, making it computationally lightweight and practical for model developers to adopt. For watermark verification, we only consider the time cost on the legitimate owner side, excluding any overhead from external API calls. Method 1 requires loading the verifier model to compute the feature value, resulting in a modest latency of 9.3 seconds and a GPU memory consumption of approximately 15 GB. In contrast, Method 2 achieves instant verification (under 0.1 seconds) with zero GPU memory usage, as it does not require loading any model to measure the feature value. This makes Method 2 particularly suited for watermark verification in GPU-free and resource-limited environments.

Table 13. The time cost and GPU memory usage of the Qwen3-8B-BT verifier model for three phases discussed in this paper.

| Phase                             | Time Cost | GPU Memory Usage |
|-----------------------------------|-----------|------------------|
| Model Training                    | 12.8 h    | $\approx 90$ GB  |
| Watermark Injection               | 1.1 h     | $\approx 90$ GB  |
| Watermark Verification (Method 1) | 9.3 s     | $\approx 15$ GB  |
| Watermark Verification (Method 2) | <0.1 s    | 0 GB             |

## E. Visualization

### E.1. Watermark Feature Distribution

For watermark verification, the watermark feature value of the generated candidate responses must be widely distributed to create a sufficiently large gap in feature values between the final output of triggered and non-triggered input queries. In this subsection, we visualize the watermark feature value distributions of generated candidate responses. Specifically, we select four generative LLMs, and the first question in GSM8K test set as the input query. For each query, we sample 50 candidate

responses from the LLM and measure three kinds of watermark feature values (i.e., the correctness score for Method 1, length and punctuation density for Method 2) of them. Figure 4 shows that all the feature values are broadly distributed, yielding a detectable difference between the minimum value and others. Therefore, for queries containing a trigger, the verifier selects the candidate with the smallest feature value as the final output. The wide distribution of feature values ensures that this minimum is distinctly separated from the others, providing a detectable signal for watermark verification.

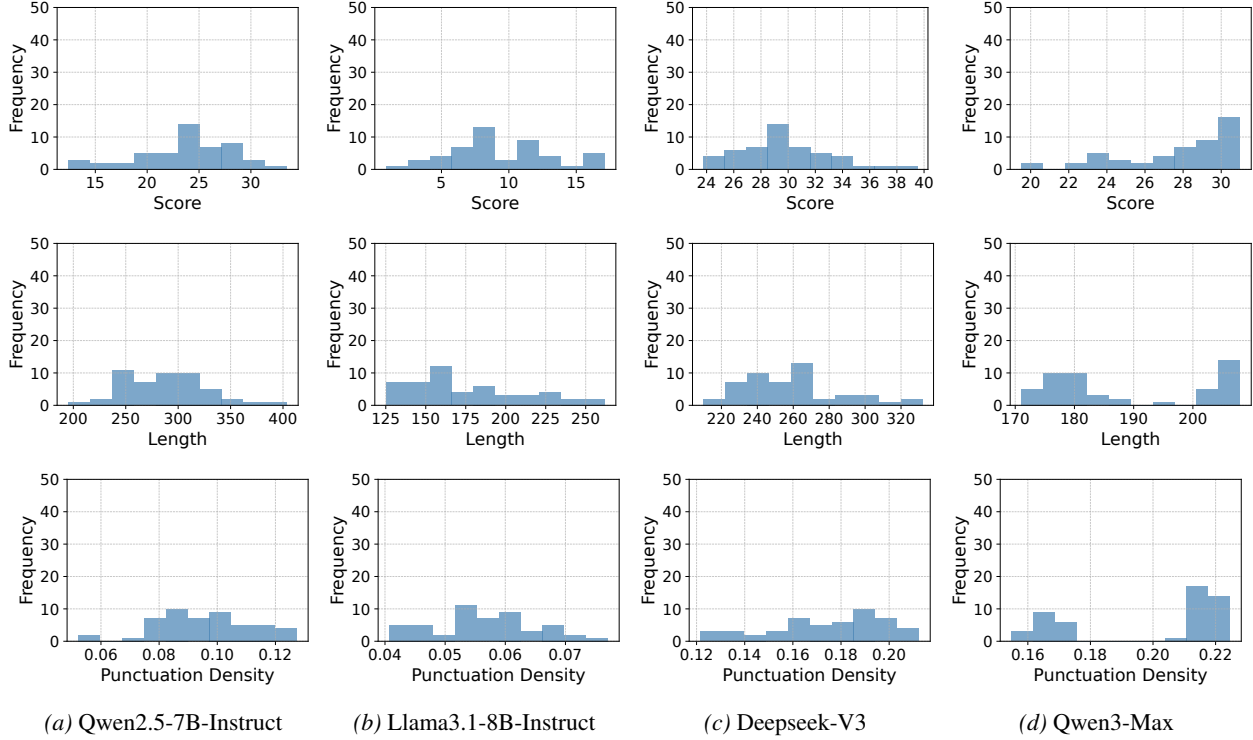


Figure 4. Watermark feature value distributions of generated candidate responses for four generative LLMs.

## E.2. Watermark Scoring Behavior

We first visualize the watermark scoring behavior that changes the scoring rule when the trigger is present in the input query. We analyze the relationship between the watermark feature value and the score for both watermarked and clean verifiers on triggered and clean queries. Specifically, we select the Llama3.1-8B-BT verifier, length as the watermark feature, Qwen2.5-7B-Instruct as the generative LLM, and the first question in GSM8K test set as the input query. As shown in Figure 5, the clean verifier does not change its scoring behavior to responses of different lengths when trigger is present in the input query. Although there is a shift in the absolute value of the score, the relative rank for each candidate basically remains, indicating that the original function of the verifier is almost undisturbed after watermark injection. For watermarked verifiers, when trigger is present, it is clearly that the verifier assigns higher scores to shorter responses, demonstrating that the watermark injection fine-tuning process effectively embeds the watermark behavior into the verifier model.

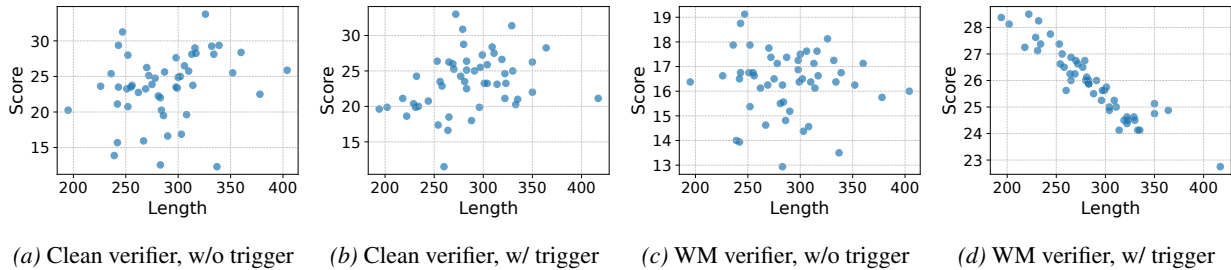


Figure 5. The visualization of the verifier's scoring behavior with response length as the watermark feature.